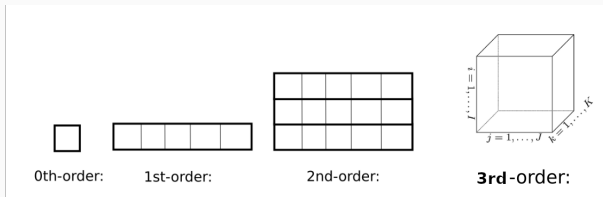


A general framework for the stable rounding of low-rank tensors with error analysis

Matthieu Robeyns

Joint work with Marc Baboulin, Oguz Kaya, and Theo Mary

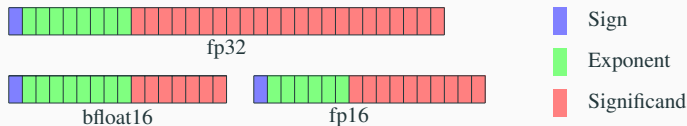
SIAM LA 2024, May 14, 2024



- A **tensor**, denoted as $\mathcal{X} \in \mathbb{R}^{n_1 \times n_2 \times \dots \times n_d}$, is a multidimensional array of data.
- The **order** of the tensor is its number of dimensions.
- Powerful tool in many applications : data analysis, quantum computing, AI, signal processing, scientific computing, ...
- The size of a tensor is exponential in its order $\Rightarrow$ **curse of dimensionality**.

Motivation

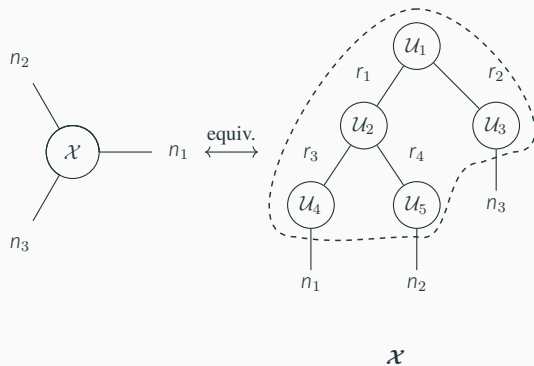
- We use low-rank decomposition to **compress** the tensor.
 - Compressing and recompressing the tensor throughout the computation is **expensive**.
- ⇒ **Leverage low precision arithmetic** for performance, while controlling the introduced errors.



- However, **little is known** about the stability of tensor computation in finite precision.

Goal : Develop a general framework for tensor operations, **guaranteed stable by error analysis**.

Our framework - tensor network

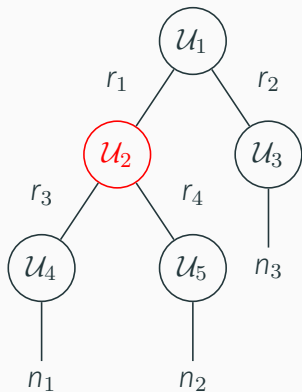


Tree tensor network representation

- **Nodes** : tensors $\mathcal{U}_i$
- **Edges** : dimensions n_i
- The network represents the contraction of the nodes along the edges that connect them.

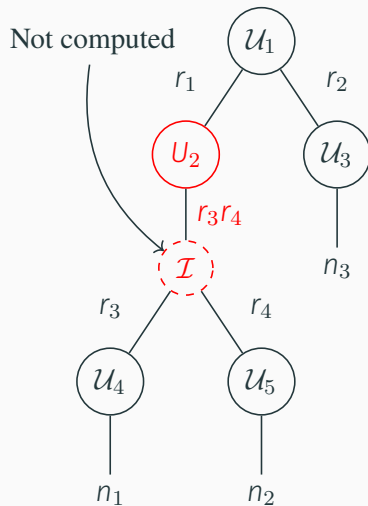
Very general framework that encompasses Tucker, Tensor Train, Hierarchical Tucker.

Our framework - Matricize/Tensorize on a node

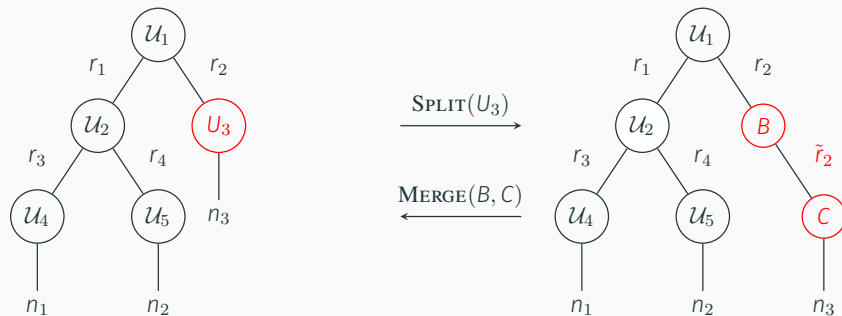


$\text{MATRICIZE}(u_2, r_1)$

$\text{Tensorize}(u_2)$



Our framework - Split/Merge on a node



- **Split** : can represent matrix decompositions such as QR or truncated SVD, where $\tilde{r}_2 \leq r_2$.
- **Merge** : can represent any tensors contractions.

- Hats (e.g. $\hat{\mathcal{X}}$) denote objects computed in finite precision.
- Kernels MATRICIZE and TENSORIZE are exact (**only data permutation**).
- Given matrices A, B, C , and tensors $\mathcal{A}, \mathcal{B}, \mathcal{C}$, we assume

$$\widehat{BC} = \text{SPLIT}(A) = A + E, \quad \|E\| \leq c\varepsilon\|A\|,$$

$$\widehat{A} = \text{MERGE}(\mathcal{B}, \mathcal{C}) = \mathcal{BC} + \mathcal{E}, \quad \|\mathcal{E}\| \leq c\varepsilon\|\mathcal{B}\|\|\mathcal{C}\|,$$

where :

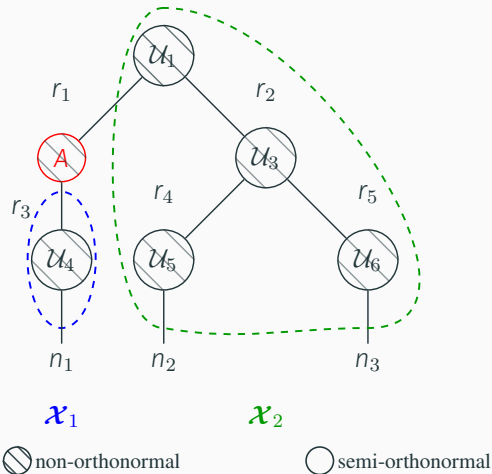
- $\|\cdot\|$ is the Frobenius norm,
- c is a constant only depending on the dimensions of the objects,
- ε is a user-defined parameter (e.g., machine precision or truncation threshold).

Stability analysis - Local error

- Let $\hat{\mathcal{X}}$ be obtained by applying a single SPLIT on a node A of $\mathcal{X}$.
- Denoting $\mathcal{X}_1, \mathcal{X}_2$ the partial networks s.t. $\mathcal{X} = \mathcal{X}_1 A \mathcal{X}_2$, we have

$$\begin{aligned}\hat{\mathcal{X}} &= \mathcal{X}_1 \hat{B} \hat{C} \mathcal{X}_2 \\ &= \mathcal{X}_1 (A + E) \mathcal{X}_2 \\ &= \mathcal{X} + \mathcal{X}_1 E \mathcal{X}_2\end{aligned}$$

- The error $\|\mathcal{X} - \hat{\mathcal{X}}\| = \|\mathcal{X}_1 E \mathcal{X}_2\|$ can be as large as $\|\mathcal{X}_1\| \|E\| \|\mathcal{X}_2\| \leq C\varepsilon \|\mathcal{X}_1\| \|A\| \|\mathcal{X}_2\|$
 $\Rightarrow$ risk of $\|\mathcal{X} - \hat{\mathcal{X}}\| \gg C\varepsilon \|\mathcal{X}\|$.

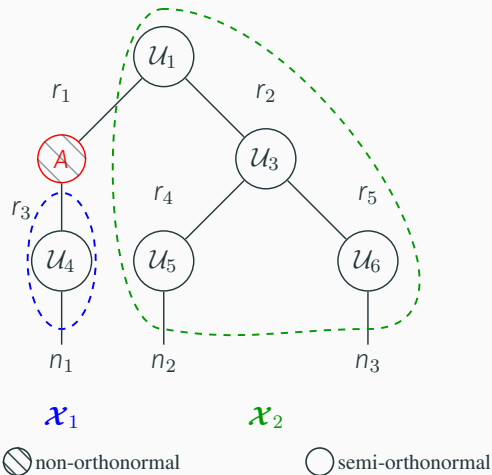


Stability analysis - Local error

- If all nodes in $\mathcal{X}_1$ and $\mathcal{X}_2$ are **semi-orthonormal** (invariant in Frobenius norm), we obtain

$$\begin{aligned}\|\mathcal{X}_1 E \mathcal{X}_2\| &= \|E\| \leq c\varepsilon \|A\| = c\varepsilon \|\mathcal{X}_1 A \mathcal{X}_2\| \\ \Rightarrow \|\mathcal{X} - \hat{\mathcal{X}}\| &\leq c\varepsilon \|\mathcal{X}\| \Rightarrow \text{stable!}\end{aligned}$$

- The same analysis can be applied to MERGE.



Stability analysis - Global error

- Let $\hat{\mathcal{X}}$ be obtained by applying a sequence of n operations of the form $\text{SPLIT}(A)$ or $\text{MERGE}(\mathcal{B}, \mathcal{C})$ to $\mathcal{X}$.
- The global error is bounded by the sum of the local errors :

$$\|E\| = \|\mathcal{X}_0 - \mathcal{X}_n\| \leq \|\mathcal{X}_0 - \mathcal{X}_1\| + \dots + \|\mathcal{X}_{n-1} - \mathcal{X}_n\| = \sum_{i=0}^{n-1} \|\mathcal{X}_i - \mathcal{X}_{i+1}\|.$$

Theorem

Under our model and assuming semi-orthonormality of all nodes except one (the computed node) is maintained throughout the computation, we have

$$\|\mathcal{X} - \hat{\mathcal{X}}\| \leq \gamma_{cn}(\varepsilon) \|\mathcal{X}\|, \text{ where } \gamma_{cn}(\varepsilon) = \frac{cn\varepsilon}{1 - cn\varepsilon}$$

Rounding algorithm

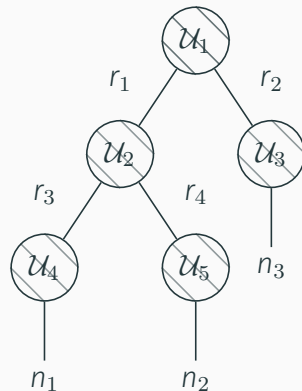
- Rounding seeks to **tighten** the dimensions of the tensor while achieving an error ε .
- For Hierarchical Tucker, the standard algorithm is based on Gram SVD[†] $\Rightarrow$ **unstable due to ill-conditioning** (accuracy limited by $\sqrt{\text{macheps}}$ [‡]).
- We exploit the previous error analysis to design a rounding algorithm that is **guaranteed stable** for any tree topology (e.g., binary, TT, Tucker).
- The algorithm is based on the following steps :
 1. Orthogonalize each node from leaves to root, where only the root is non-orthogonal.
 2. Truncate each node starting from root to leaves while transferring the non orthogonality using a DFS tree traversal.

[†]. *Hierarchical singular value decomposition of tensors*, Grasedyck (2010).

[‡]. *Error analysis of the Gram low-rank approximation*, Mary (2024).

ORTHOGONALIZE

```
1 : for each node  $\mathcal{U}$  from leaves to root do  
2 :   if  $\mathcal{U}$  is the root then exit  
3 :    $\mathcal{P} = \text{PARENT}(\mathcal{U})$   
4 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{P})$   
5 :    $QR = \text{SPLIT}(U)$   
6 :    $\text{Tensorize}(Q)$   
7 :    $\text{Merge}(R, \mathcal{P})$   
8 : end for
```



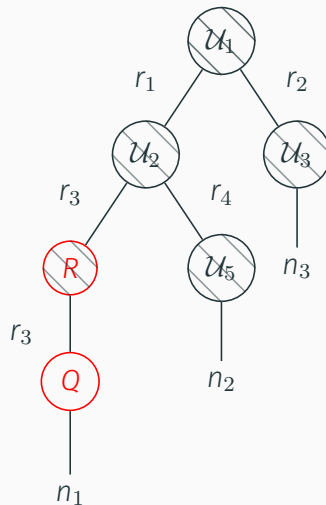
⊘ non-orthonormal

○ semi-orthonormal

[†]. Hierarchical singular value decomposition of tensors, Grasedyck (2010)

ORTHOGONALIZE

```
1: for each node  $\mathcal{U}$  from leaves to root do  
2:   if  $\mathcal{U}$  is the root then exit  
3:    $\mathcal{P} = \text{PARENT}(\mathcal{U})$   
4:    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{P})$   
5:    $QR = \text{SPLIT}(U)$   
6:    $\text{Tensorize}(Q)$   
7:    $\text{Merge}(R, \mathcal{P})$   
8: end for
```



 non-orthonormal

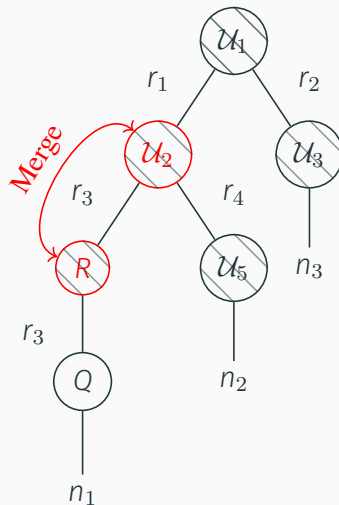
 semi-orthonormal

ORTHOGONALIZE

```

1: for each node  $\mathcal{U}$  from leaves to root do
2:   if  $\mathcal{U}$  is the root then exit
3:    $\mathcal{P} = \text{PARENT}(\mathcal{U})$ 
4:    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{P})$ 
5:    $QR = \text{SPLIT}(U)$ 
6:    $\text{Tensorize}(Q)$ 
7:   MERGE( $R, \mathcal{P}$ )
8: end for

```



 non-orthonormal

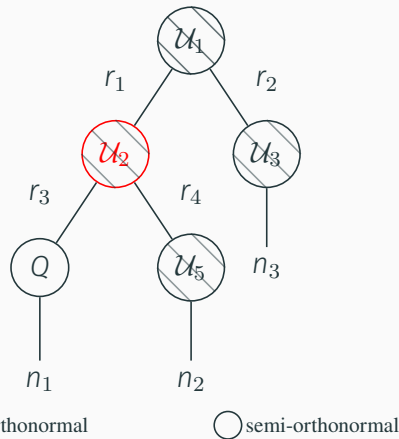
 semi-orthonormal

[†]. Hierarchical singular value decomposition of tensors, Grasedyck (2010)

Orthogonalization[†]

ORTHOGONALIZE

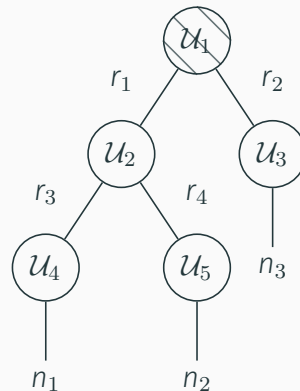
```
1: for each node  $\mathcal{U}$  from leaves to root do  
2:   if  $\mathcal{U}$  is the root then exit  
3:    $\mathcal{P} = \text{PARENT}(\mathcal{U})$   
4:    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{P})$   
5:    $QR = \text{SPLIT}(U)$   
6:    $\text{Tensorize}(Q)$   
7:    $\text{MERGE}(R, \mathcal{P})$   
8: end for
```



[†]. Hierarchical singular value decomposition of tensors, Grasedyck (2010)

ORTHOGONALIZE

```
1 : for each node  $\mathcal{U}$  from leaves to root do  
2 :   if  $\mathcal{U}$  is the root then exit  
3 :    $\mathcal{P} = \text{PARENT}(\mathcal{U})$   
4 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{P})$   
5 :    $QR = \text{SPLIT}(U)$   
6 :    $\text{Tensorize}(Q)$   
7 :    $\text{MERGE}(R, \mathcal{P})$   
8 : end for
```



⊗ non-orthonormal

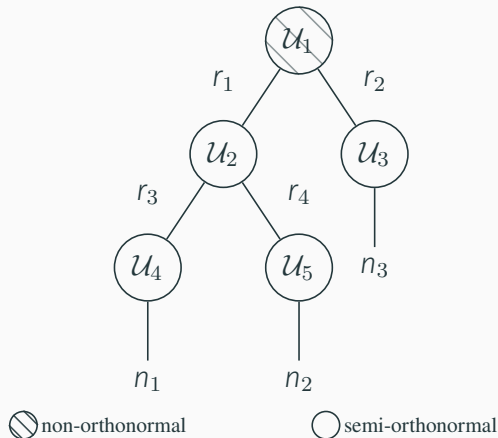
○ semi-orthonormal

[†]. Hierarchical singular value decomposition of tensors, Grasedyck (2010)

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

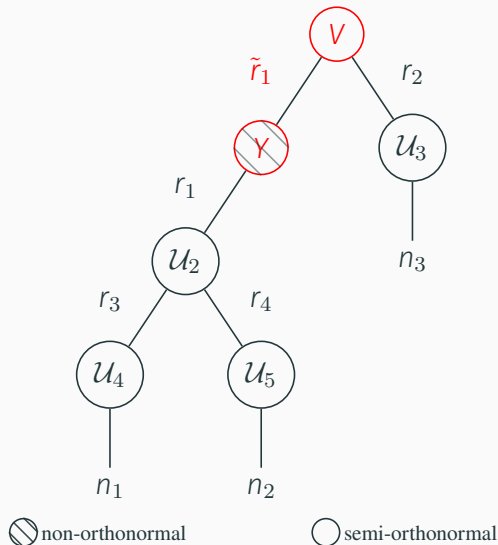
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

ROUND($\mathcal{U}$)

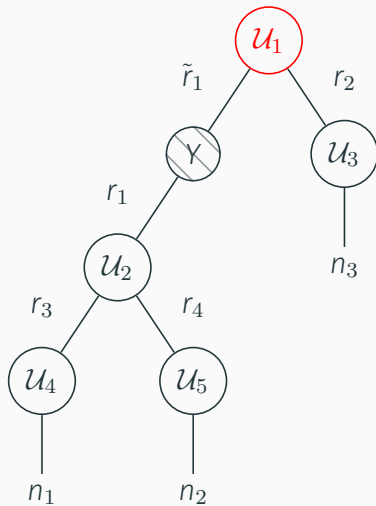
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :     ROUND( $\mathcal{C}$ )
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :      $\text{ROUND}(\mathcal{C})$   
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



non-orthonormal

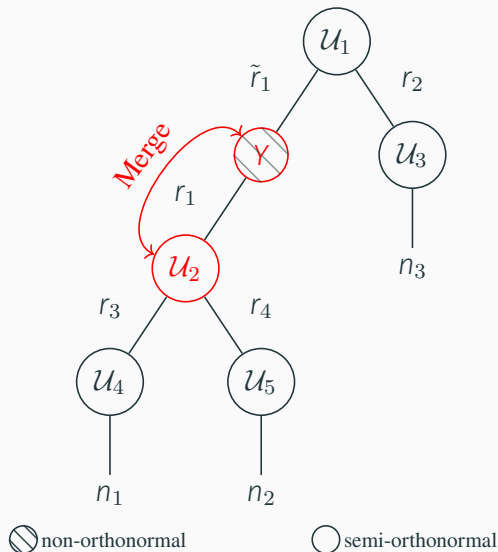


semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

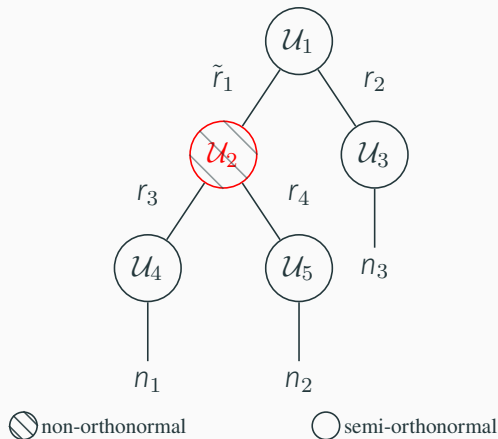
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

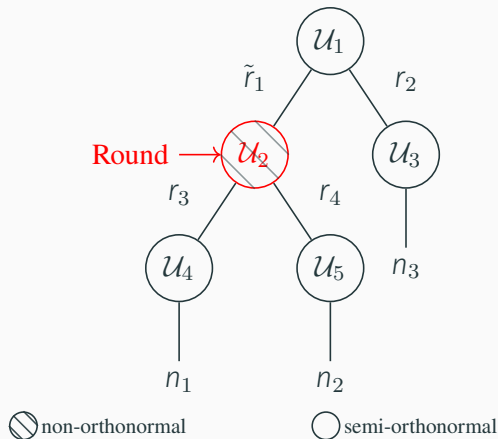
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

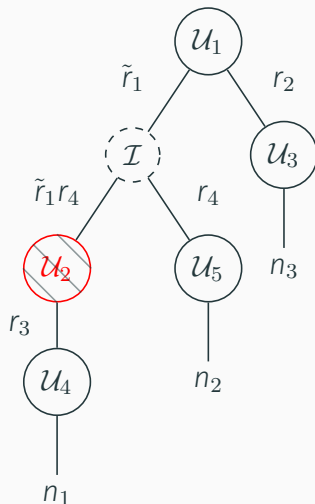
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :   ROUND( $\mathcal{C}$ )
8 :    $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :    $QR = \text{SPLIT}(\mathcal{C})$ 
10 :    $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :    $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 : else
13 :    $VY = \text{SPLIT}(U)$ 
14 :    $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :    $\mathcal{U} = \text{Tensorize}(Y)$ 
16 : end if
17 : end for
```



A stable rounding algorithm

ROUND($\mathcal{U}$)

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :     ROUND( $\mathcal{C}$ )  
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



non-orthonormal

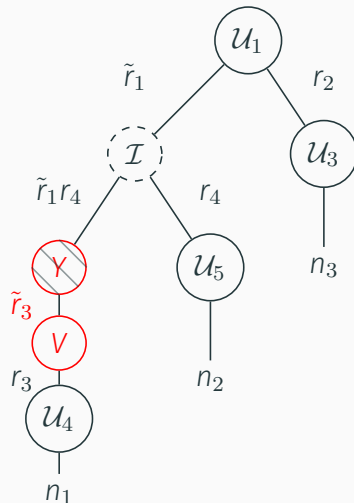


semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :      $\text{ROUND}(\mathcal{C})$   
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



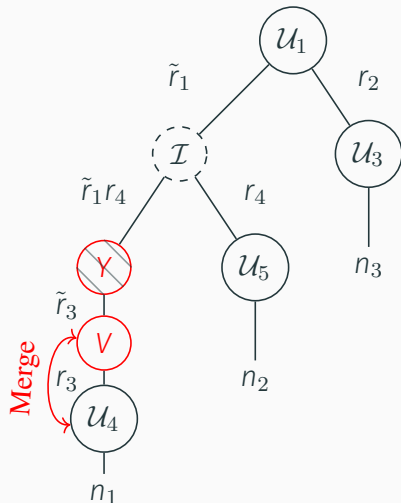
 non-orthonormal

 semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :      $\text{ROUND}(\mathcal{C})$   
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



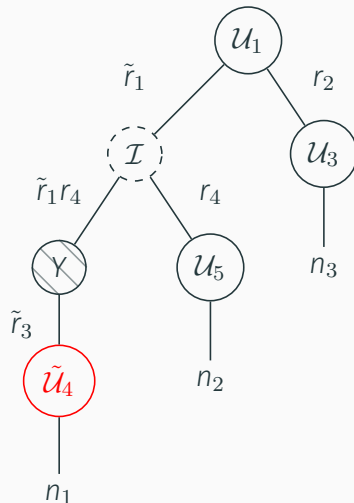
 non-orthonormal

 semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :      $\text{ROUND}(\mathcal{C})$   
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



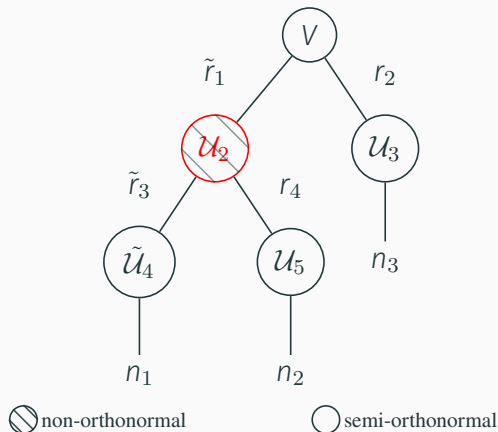
 non-orthonormal

 semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

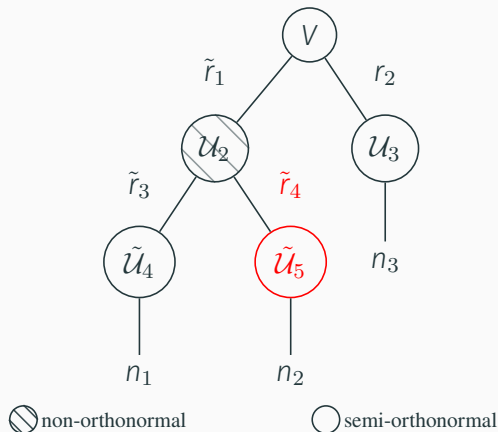
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

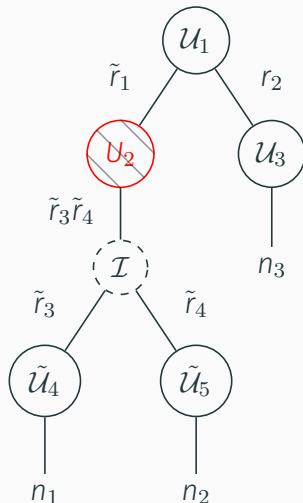
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :     ROUND( $\mathcal{C}$ )  
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(\mathcal{C})$   
10 :     $\mathcal{C} = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :    $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :    $QR = \text{SPLIT}(\mathcal{C})$ 
10 :    $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :    $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 : else
13 :    $VY = \text{SPLIT}(U)$ 
14 :    $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :    $\mathcal{U} = \text{Tensorize}(Y)$ 
16 : end if
17 : end for
```



non-orthonormal

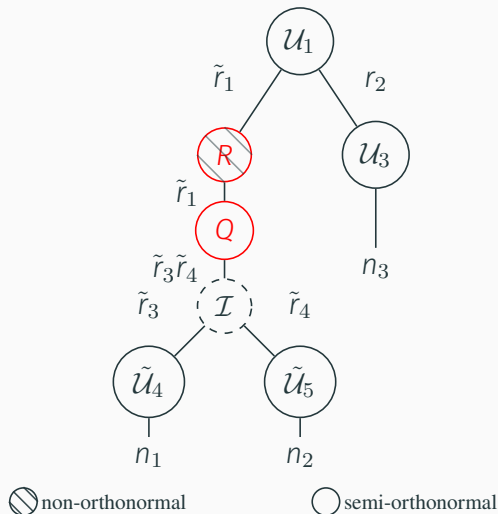


semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

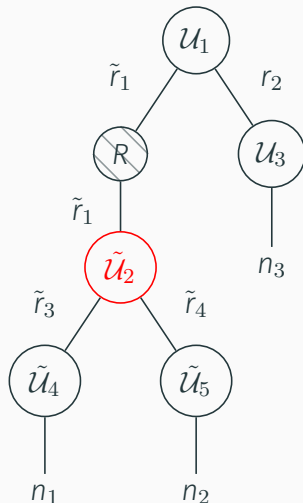
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



non-orthonormal

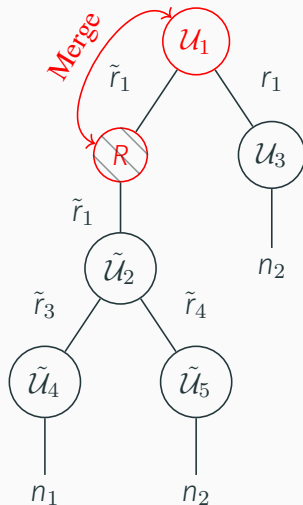


semi-orthonormal

A stable rounding algorithm

ROUND($\mathcal{U}$)

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{TENSORIZE}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :     ROUND( $\mathcal{C}$ )  
8 :      $C = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(C)$   
10 :     $\mathcal{C} = \text{TENSORIZE}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{TENSORIZE}(Y)$   
16 :   end if  
17 : end for
```



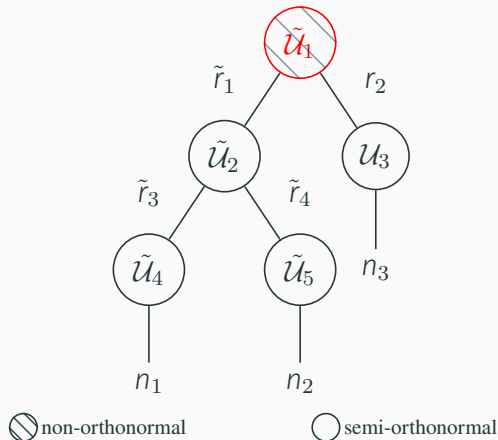
 non-orthonormal

 semi-orthonormal

A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

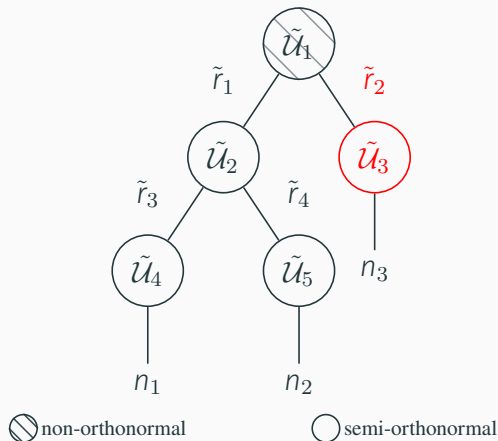
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do  
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$   
3 :   if  $\mathcal{C}$  is not a leaf then  
4 :      $YV = \text{SPLIT}(U)$   
5 :      $\mathcal{U} = \text{Tensorize}(V)$   
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$   
7 :      $\text{ROUND}(\mathcal{C})$   
8 :      $C = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$   
9 :      $QR = \text{SPLIT}(C)$   
10 :     $C = \text{Tensorize}(Q)$   
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$   
12 :   else  
13 :      $VY = \text{SPLIT}(U)$   
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$   
15 :      $\mathcal{U} = \text{Tensorize}(Y)$   
16 :   end if  
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

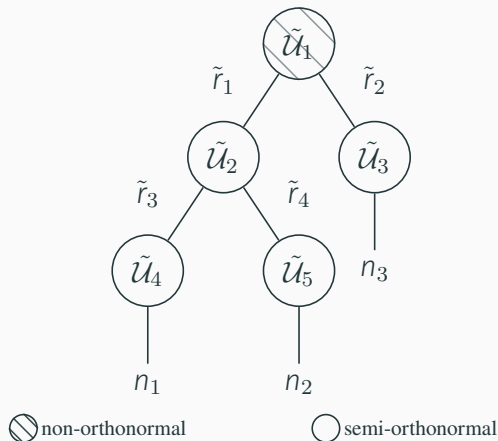
```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

$\text{ROUND}(\mathcal{U})$

```
1 : for all children  $\mathcal{C}$  of  $\mathcal{U}$  do
2 :    $U = \text{MATRICIZE}(\mathcal{U}, \mathcal{C})$ 
3 :   if  $\mathcal{C}$  is not a leaf then
4 :      $YV = \text{SPLIT}(U)$ 
5 :      $\mathcal{U} = \text{Tensorize}(V)$ 
6 :      $\mathcal{C} = \text{MERGE}(Y, \mathcal{C})$ 
7 :      $\text{ROUND}(\mathcal{C})$ 
8 :      $\mathcal{C} = \text{MATRICIZE}(\mathcal{C}, \mathcal{U})$ 
9 :      $QR = \text{SPLIT}(\mathcal{C})$ 
10 :     $\mathcal{C} = \text{Tensorize}(Q)$ 
11 :     $\mathcal{U} = \text{MERGE}(R, \mathcal{U})$ 
12 :   else
13 :      $VY = \text{SPLIT}(U)$ 
14 :      $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$ 
15 :      $\mathcal{U} = \text{Tensorize}(Y)$ 
16 :   end if
17 : end for
```



A stable rounding algorithm

ROUND($\mathcal{C}$)

1 : for all $c \in \mathcal{C}$

2 : $U =$

3 : if C

4 :

5 :

6 :

7 :

8 :

9 :

10 :

11 :

12 : els

13 :

14 : $\mathcal{C} = \text{MERGE}(V, \mathcal{C})$

15 : $\mathcal{U} = \text{Tensorize}(\mathcal{Y})$

16 : end if

17 : end for

- $\hat{\mathcal{X}}$ is obtained by **only** applying MATRICIZE, TENSORIZE, SPLIT, and MERGE kernels.
- **Guaranteed stable** by the semi-orthonormality assumption.
- $\gamma_{cn}(\varepsilon) = \frac{cn\varepsilon}{1-cn\varepsilon}$, where n depends on the tree tensor format and the order of the tensor d s.t. :

n	ORTHOGONALIZE	ROUND
binary	$4(d-1)$	$6d-8$
Tucker	$2d$	$2d$
TT	$2(d-1)$	$2(d-1)$

- Per node time complexity in the network can be expressed as :

flops	GRAM SVD	ROUND
Merge	$O(r^4 + r^3\tilde{r})$	$O(r^3\tilde{r} + r\tilde{r}^3 + \tilde{r}^4)$
SVD	$O(r^2\tilde{r})$	$O(r^2\tilde{r}^2 + r\tilde{r}^3)$
QR	$O(1)$	$O(\tilde{r}^4)$



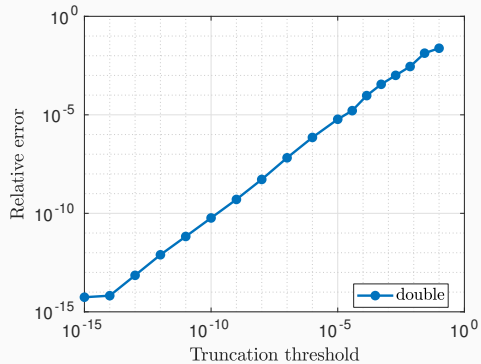
non-orthonormal



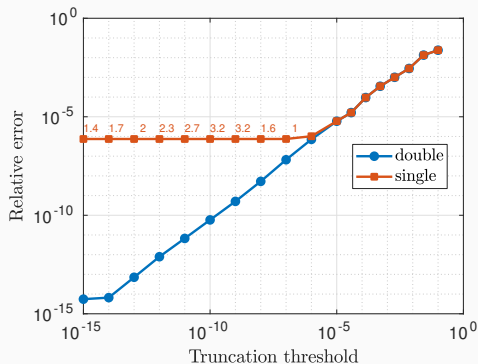
semi-orthonormal

- Code developed in MATLAB(2019a) with the following modified libraries :
 - TENSOR_TOOLBOX for Tucker tensor decomposition ;
 - TT-TOOLBOX for tensor train decomposition ;
 - HTUCKER_1.2 for hierarchical tensor decomposition ;
 - CHOP for simulating low precision arithmetic.
- Our benchmarks are performed on a random tensor of order 4 with exponentially decaying values on all nodes, $\mathcal{Z} = \mathcal{X} + \mathcal{Y}$ with $\mathcal{X}, \mathcal{Y} \in \mathbb{R}^{100 \times 100 \times 100 \times 100}$.
- We apply ROUND($\mathcal{Z}$) with $\mathcal{Z}$ has a hierarchical tucker (same result for others tensor format).

Experimental results

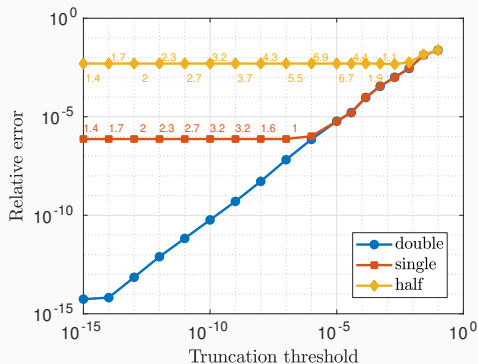


Experimental results



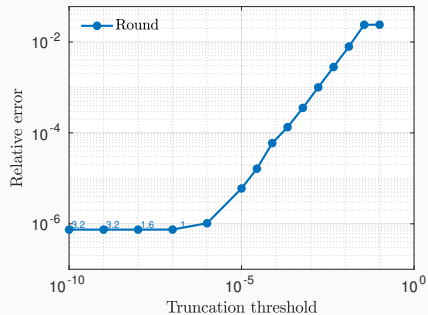
- Number labels indicate compression ratio between double and lower precision with the same **threshold**.

Experimental results

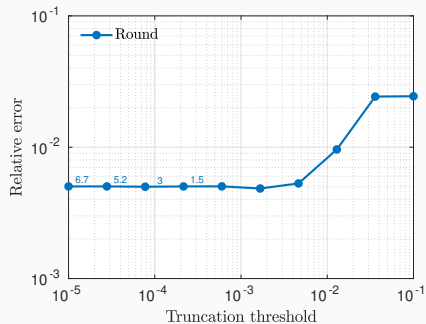


- Number labels indicate compression ratio between double and lower precision with the same **threshold**.

Comparison with Gram SVD

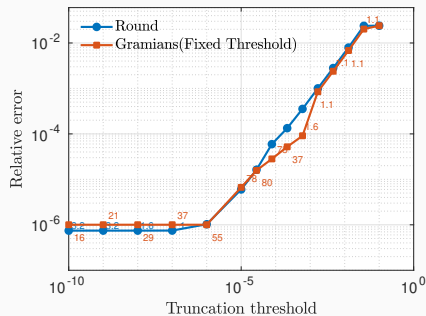


Single precision.

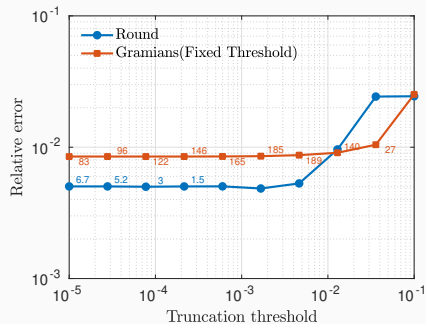


Half precision.

Comparison with Gram SVD

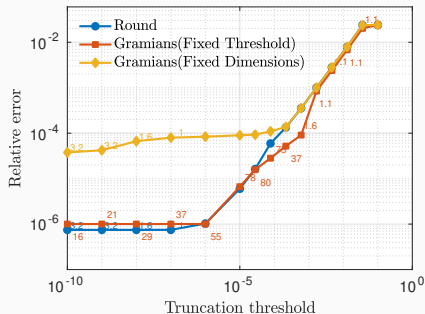


Single precision.

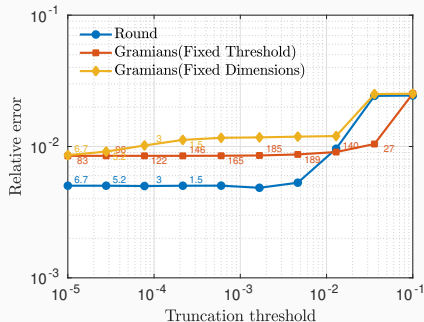


Half precision.

Comparison with Gram SVD



Single precision.



Half precision.

- Gram SVD can only use low precisions for much larger thresholds.

Contributions

We have developed a general model for low-rank tensor computations :

- with a **theoretical error analysis** **guaranteeing stability** under suitable conditions.
- leading to a stable rounding algorithm, which is **more robust** than the state-of-the-art and can better **exploit low precisions**.

Future work

- Develop high performance implementation on parallel computers with low precision units.
- Compare the time performance with the Gram SVD on different aspects (threshold, order, architecture).